Sensor Investigation Brief

Test Engineering Assignment | COMPSYS 732

Sensors: Wheel Odometry | 2D LiDAR | RGB Camera (OAK-D)

1. Your Role

In the previous five tasks you learned how to communicate with the TurtleBot 4 in simulation; exploring ROS 2 topics with CLI tools, visualising sensor data in RViz, and writing Python nodes that read sensor data and command autonomous motion.

You will now apply those skills to the physical robot. Your role is that of a Test Engineer responsible for characterising the performance of three onboard sensors: wheel odometry, 2D LiDAR, and the OAK-D camera. A Test Engineer does not just operate a system, they design repeatable experiments, collect quantitative evidence, and draw defensible conclusions about whether a sensor or device meets the requirements of reliable operation.

You need to conduct structured experiments on the real TurtleBot 4 in the lab environment (take it as a warehouse), collect data, and build up a set of results and plots. At the end of the investigation period, you will attend an invigilated report session where you will write up your findings.

> **Report Note**
>
> Your plots, CSV files, and results tables are your evidence. Without them you cannot write the report. Treat your data carefully and save it to multiple locations.

2. Connecting to Your Simulation Skills

The experiments in this brief are not standalone activities; they are direct extensions of what you built in the five simulation tasks.

> **Key difference from simulation**
>
> When you teleoperate, every run is slightly different. Your test nodes must command identical motion every trial so that differences in sensor readings are due to the sensor, not your driving.

3. Writing Your Test Nodes

Each experiment requires an autonomous test node; a Python ROS 2 node that commands a precise, repeatable motion sequence and stops cleanly at the end. This section assumes you have completed the **Before You Start setup** document in Canvas; if you have not done so, do that first before continuing here.

The template is already in your package if you completed Task 5. Check first:

```
ls ~/ros2_ws/src/tb4_sensor_reader/tb4_sensor_reader/test_node_template.py
```

If the file is listed, skip to the NAMESPACE step below. If not found, copy it in from lab_scripts:

```
cp ~/ros2_ws/src/lab_scripts/test_node_template.py \
   ~/ros2_ws/src/tb4_sensor_reader/tb4_sensor_reader/test_node_template.py
```

Verify the copy worked before continuing:

```
ls ~/ros2_ws/src/tb4_sensor_reader/tb4_sensor_reader/test_node_template.py
```

The template already has the publisher and subscriber structure wired up, a phase-based control loop running at 10 Hz, and helper functions for driving and stopping. You fill in the TODO sections with your specific motion sequence.

Calculating motion durations

Because the robot runs open-loop (no feedback on exact position during the motion), you must calculate how long to apply each command to achieve a target displacement or rotation, for example:

Motion	Formula	Example at default speeds
Drive straight N metres	duration = N / FORWARD_SPEED	1.0 m at 0.2 m/s → 5.0 s
Turn N degrees	duration = (N × π/180) / TURN_SPEED	90° at 0.5 rad/s → 3.14 s
Full 360° rotation	duration = (2π) / TURN_SPEED	360° at 0.5 rad/s → 12.57 s

Adding your node to setup.py

After copying and editing the template, register it in setup.py and rebuild:

```
# In setup.py entry_points console_scripts:
'test_node = tb4_sensor_reader.test_node_template:main',
```

After any edit to the template, rebuild and source the workspace; this is required every time and in every new terminal:

```
cd ~/ros2_ws && colcon build --packages-select tb4_sensor_reader && source ~/ros2_ws/install/setup.bash
```

Phase numbering

The template uses a phase counter (self.phase) to sequence motion steps. If you remove a phase, renumber all subsequent phases so they are consecutive starting from 0; otherwise the node will run Phase 0 and then hang waiting for a phase number that never arrives.

Also: `NAMESPACE` must be a string in quotes with a forward slash; e.g. `'/T8'` not `T8` or `/T8`. Incorrect formatting causes a Python error on startup.

```
cd ~/ros2_ws
colcon build --packages-select tb4_sensor_reader
source ~/ros2_ws/install/setup.bash
```

Running a test node alongside a logger

Most experiments require two terminal tabs running simultaneously; one running your test node (which commands motion) and one running the logger script (which records data). Open both tabs before starting, then follow this sequence:

1. In Tab 1, start the logger; it will print a waiting message and stay open (you have 30 seconds to switch tabs):

```
~/ros2_venv/bin/python3 ~/ros2_ws/src/lab_scripts/odom_logger.py --namespace /TXX --mode linear --target 1.0 --duration 30 --trial 1
```

2. Switch to Tab 2 and run the test node immediately after the logger shows its waiting message:

```
source ~/ros2_ws/install/setup.bash
ros2 run tb4_sensor_reader test_node
```

Every new terminal tab needs sourcing

Run `source ~/ros2_ws/install/setup.bash` in every new terminal tab before running ros2 commands. This is not done automatically on the lab PCs. If you see `Package not found` or `No executable found`, this is almost always the reason.

If you switch to a different lab PC during the session, you must also rebuild first: cd ~/ros2_ws && colcon build --packages-select tb4_sensor_reader && source ~/ros2_ws/install/setup.bash

4. Statistical Analysis

Observing a single measurement tells you very little. Repeating an experiment multiple times and computing statistics tells you how consistent and accurate the sensor is. The following metrics are examples that you can use in your report, you are not limited to these:

Metric	Formula	What it tells you
Mean ($\bar{x}$)	$\bar{x} = (x_1 + x_2 + ... + x_n) / n$	The average result; how close the sensor is to truth on average
Standard Deviation (σ)	$\sigma = \text{sqrt}(\Sigma(x_i - \bar{x})^2 / n)$	How consistent the readings are across trials; sensor precision
Error	error = $\bar{x}$ − ground truth	Signed difference between average reading and known true value
% Error	% error = |error| / ground truth × 100	Normalised accuracy; allows comparison across different distances

Note

The logger scripts compute these metrics automatically and print them to the terminal after each trial. They also append each trial's result to a cumulative CSV file (odom_stats.csv, lidar_range_stats.csv, detections.csv) so you can track results across all trials in one place.

Pass / Fail criteria

As a Test Engineer you must define your own pass/fail criteria before collecting data; not after. Think about what level of accuracy would be acceptable for a mobile robot operating in a warehouse environment with the dimensions of the lab.

Tip

A robot navigating a 5m corridor needs much tighter odometry accuracy than one crossing a car park. The pass/fail threshold should reflect a realistic use case, not just whatever your robot happens to achieve.

5. Investigation A; Wheel Odometry

Wheel odometry estimates the robot's position by counting encoder pulses and integrating velocity over time. It is the primary localisation source for the TurtleBot 4 when no external reference is available. Your investigation must characterise two properties: linear displacement accuracy over a known distance, and accumulated drift over a closed-loop path.

Test A1; Linear Displacement Accuracy

Design a test node that drives the robot straight forward for a fixed speed and duration corresponding to a target distance.

Important

Make sure the robot starts from exactly the same position and orientation each trial. Small misalignments at the start compound into large errors. Use a mark and align the robot carefully if needed.

You already built an odometry subscriber in Task 4; Test A1 is a direct application of that work.

Your node must:

- Subscribe to /TXX/odom
- When the robot stops, print the final `pose.pose.position.x` and `pose.pose.position.y` values to the terminal
- Compute and print displacement
- Record that printed value in Results Table A1

You may use odom_logger.py for the square test (Test A2) and for generating the trajectory plot; but for Test A1 the subscriber node is your own.

Example of the results tables:

Pass/Fail criterion: % error < _____ %

Trial	Target (m)	Odom reported (m)	Physical measurement (m)	Error (m)	% Error	Pass / Fail

Summary statistics:

Metric	Value
Mean displacement (m)	
Std dev (m)	
Mean % error	

Test A2; Closed-Loop Drift

Extend your test node to drive a square and stop. The robot should end up back at the start. The odometry closing error, the distance between the reported final position and the origin, is your drift metric.

```
~/ros2_venv/bin/python3  ~/ros2_ws/src/lab_scripts/odom_logger.py  --namespace
/TXX --mode square --duration 60 --trial 1
```

Example: Pass/Fail criterion: closing error < _____ m

Trial	Odom closing error (m)	Physical closing error (m)	Final heading error (deg)	Pass / Fail

6. Investigation B; 2D LiDAR

The TurtleBot 4 uses an RPLIDAR-A1 sensor that produces a 360-degree ring of distance measurements. Your investigation must characterise range accuracy at known distances and measurement consistency.

Test B1; Range Accuracy at Known Distances

Position the robot at different distances facing a wall. At each position, run the logger in range mode to collect 50 stationary readings. The logger will compute mean, std dev, and % error automatically.

Figure out the front of the lidar. Verify alignment with a snapshot before collecting range data.

```
# Example for 1.0 m, trial 1
~/ros2_venv/bin/python3  ~/ros2_ws/src/lab_scripts/lidar_logger.py  --namespace
/TXX --mode range --ground-truth 1.0 --samples 50 --trial 1
```

Example; Results Table B1; Range Accuracy

Ground truth (m)	Trial	Mean reading (m)	Std Dev (m)	Error (m)	% Error	Pass / Fail

Test B2; Environment Snapshot

For this test you will write a short subscriber node rather than using the logger script. This gives you direct experience of reading and interpreting a raw LaserScan message.

Create a new Python node (or add to my_test_node.py) that:

1. Subscribes to /TXX/scan
2. In the callback, extracts and prints: the total number of beams (`len(msg.ranges)`), the minimum valid range (`min(r for r in msg.ranges if msg.range_min <= r <= msg.range_max)`), and the range at the index corresponding to straight ahead
3. Prints these three values once per scan, then keeps running until Ctrl+C

Place the robot on the floor, run your node, and observe how the readings change as you manually rotate the robot. Record the minimum and maximum valid range you observe in your results table below.

Once you have done this, you may also run `lidar_logger.py --mode snapshot` to generate the polar plot PNG for your report; but the printed values must come from your own node.

An example of Results Table B2; Environment Snapshot

Observation	Value	Unit	Notes
Total beams per scan		beams	Should be ~360
Minimum valid range observed		m	Closest obstacle in lab
Maximum valid range observed		m	Open direction
Range straight ahead (180°)		m	Robot camera side facing wall or open space
Any anomalies noted (gaps, inf values)		—	Describe in Notes

Note

Save lidar_snapshot.png and lidar_range_stats.csv.

7. Investigation C; RGB Camera (OAK-D)

The TurtleBot 4 Standard is equipped with an OAK-D camera that captures a colour RGB image. Your investigation uses this camera to detect a red object in the scene using HSV colour thresholding; a fundamental technique used in robot vision for object detection and tracking.

Verify camera topic

Run `ros2 topic hz /TXX/oakd/rgb/image_raw/compressed` should show ~5 Hz. If rate is 0 and robot is undocked, dock and undock again to restart the OAK-D node.

Test C1; Live View and Observations

Verify the camera feed is working by opening a live view window. Point the camera at a variety of objects and surfaces and note what the camera can and cannot see clearly.

```
~/ros2_venv/bin/python3 ~/ros2_ws/src/lab_scripts/camera_logger.py --namespace
/TXX --mode view
```

Test C2; Red Object Detection

Place a red object in front of the camera at different distances. Run the detect mode to see the live detection overlay and observe how the pixel count changes with distance. The script saves a snapshot and CSV entry each time the object is detected above the threshold.

```
~/ros2_venv/bin/python3 ~/ros2_ws/src/lab_scripts/camera_logger.py --namespace
/TXX --mode detect
```

Example of Results Table C2; Detection Observations

Distance (m)	Surface / Object	Red pixels detected	Object visible? (Y/N)	Notes

Note

Save detection_N.png snapshots and detections.csv.

8. Invigilated Report Guidance

At the end of the investigation period, you will attend an invigilated report session. You may bring your digital plots and completed results tables. You will write the report during the session.

What to bring

- All plots generated as you see fit (PNG files)
- Your results tables
- Your cumulative stats CSV files (odom_stats.csv, lidar_range_stats.csv, detections.csv)
- Your test node and subscriber node source code; you may be asked to explain design decisions

Important

The report assesses your engineering judgement, not your ability to reproduce numbers. You must be able to explain why your results look the way they do; not just report them.

Quick Reference

Robot setup commands

```
set-turtlebot XX          # configure ROS for robot number XX
ros2 action send_goal /TXX/undock irobot_create_msgs/action/Undock {}
sanity                    # verify environment variables
ros2 topic list           # confirm robot topics are visible
ros2 topic hz /TXX/odom   # check odometry publish rate
ros2 topic hz /TXX/scan   # check LiDAR publish rate
```

Logger script summary

Script	Mode	Command
odom_logger.py	linear	~/ros2_venv/bin/python3 odom_logger.py --namespace /TXX --mode linear --target 1.0 --duration 30 --trial N
odom_logger.py	square	~/ros2_venv/bin/python3 odom_logger.py --namespace /TXX --mode square --duration 60 --trial N
lidar_logger.py	snapshot	~/ros2_venv/bin/python3 lidar_logger.py --namespace /TXX --mode snapshot
lidar_logger.py	range	~/ros2_venv/bin/python3 lidar_logger.py --namespace /TXX --mode range --ground-truth D --samples 50 --trial N
camera_logger.py	view	~/ros2_venv/bin/python3 camera_logger.py --namespace /TXX --mode view
camera_logger.py	detect	~/ros2_venv/bin/python3 camera_logger.py --namespace /TXX --mode detect

Output files

File	Contents	When generated
odom_linear_tN.png	XY trajectory + velocity + heading plots	After each linear trial
odom_square_tN.png	XY trajectory showing closing error	After each square trial
odom_stats.csv	Cumulative stats across all odom trials	Appended after every odom run
lidar_snapshot.png	360° polar plot of the lab environment	Snapshot mode
lidar_range_gtX_tN.png	Histogram of range readings vs ground truth	After each range trial
lidar_range_stats.csv	Cumulative stats across all LiDAR range trials	Appended after every range run
detection_N.png	Snapshot saved when red object detected above threshold	Detect mode
detections.csv	CSV log of all detections with pixel count and timestamp	Detect mode